



Birzeit University
Faculty of Engineering and Technology
Department of Electrical and Computer Engineering

SecureChannel

A Secure Communication Application Report

Team Members

[Jibreel Bornat - 1230872]

[Amjad Qaher - 1230800]

Repository: <https://github.com/jibreel1122/JibreelAmjadSecureChannel>

Date: June 2026

Contents

2. Introduction	3
3. System Architecture	3
3.1 Phase A - Handshake	3
3.2 Phase B - Secure Messaging	3
3.3 Code Layout	4
4. Cryptographic Primitives	4
5. Protocol Design	5
5.1 Handshake	5
5.2 Key Schedule	6
5.3 Message Format	7
Message header	7
Wire frame	7
5.4 Nonce Construction	7
5.5 Associated Data	8
5.6 Replay Protection	8
6. Security Analysis	9
6.1 Properties Achieved	9
6.2 Known Limitations	9
7. Testing	9
8. How to Run	10
9. References	10
10. AI Usage Statement	11
11. Individual Contribution Statements	11

2. Introduction

SecureChannel is a secure communication application implemented from scratch in pure Python for the ENCS4320 Applied Cryptography course project. Two parties a client and a server communicate over a TCP socket on localhost or a LAN. A session has two phases: a handshake phase that establishes a shared secret and mutually authenticates the two parties, and a secure messaging phase in which every application message is protected with an AEAD cipher and carries a plaintext, authenticated header with replay protection.

All cryptographic primitives used by the protocol SHA-256, HMAC-SHA256, HKDF, ChaCha20, Poly1305, the ChaCha20-Poly1305 AEAD construction, and X25519 were implemented directly from their official RFC/FIPS specifications, without relying on any cryptographic library inside the implementation itself. Cryptographic libraries were used only in the test suite, to cross-check our outputs against a trusted reference. This report describes the resulting protocol design: the message formats, the key schedule, the nonce construction, and the mechanisms used to prevent replay and tampering, together with the security properties and known limitations of the implementation.

3. System Architecture

The system is organized into two sequential phases, implemented across five code layers.

3.1 Phase A - Handshake

Each party generates an ephemeral X25519 key pair and the two ephemeral public keys are exchanged. Both sides then build an identical handshake transcript, authenticate it under a pre-shared key (PSK) using HMAC, and if authentication succeeds derive a set of directional session keys and nonce bases from the X25519 shared secret using HKDF. A mismatching authentication tag aborts the connection immediately.

3.2 Phase B - Secure Messaging

Once the handshake completes, every application message is wrapped in a length-prefixed wire frame consisting of a plaintext, authenticated header and a ChaCha20-Poly1305-encrypted payload. The receiver checks both the authentication tag and a strictly increasing per-direction sequence number before accepting a message.

3.3 Code Layout

The implementation is organized into five layers:

Layer	Directory	Contents
Primitives	crypto/	SHA-256, HMAC-SHA256, HKDF, ChaCha20, Poly1305 (+ one-time key generation), ChaCha20-Poly1305 AEAD, X25519 each implemented from its RFC/FIPS specification.
Protocol	protocol/	Handshake message wrapper (a_handshake.py, a_symmetricstate.py, a_cipherstate.py), transcript construction (j_transcript.py), key schedule (j_key_schedule.py), message header/frame format (j_message.py), ephemeral keypair generation (a_keypair.py).
Networking	network/	Client (j_client.py) and server (a_server.py): TCP socket handling, length-prefixed framing, the command-line chat interface, and the encrypt/decrypt and replay-check logic for the secure channel.
Bonus UI	bonus/	Tkinter GUI for the client (j_gui.py) and server (j_server_gui.py), plus a launcher that starts both together (j_launcher.py). The GUIs call the same real protocol functions as the CLI. The client GUI additionally includes two deliberate attacker-simulation buttons, “Send tampered tag” and “Send tampered header,” which corrupt one byte to demonstrate that the receiver rejects tampered tags and tampered headers while the connection stays alive.
Tests	test/	One test module per primitive, each checked against the official RFC/FIPS test vectors.

The Noise style message wrapper in protocol/a_handshake.py is used only as a transport for the two ephemeral X25519 public keys during the handshake. Its internal key-derivation output (the result of its split() step) is not used to secure the channel the actual session keys come from the explicit HKDF-based key schedule described in Section 5.2.

4. Cryptographic Primitives

Each primitive below is implemented entirely from its standards document; no cryptographic library is imported inside any crypto/ module.

SHA-256 (FIPS 180-4, *crypto/j_sha256.py*). Implements the standard padding rule (a single 0x80 byte, zero padding to 56 mod 64, a 64-bit big-endian length field), the FIPS-specified round constants and initial hash values, and the 64-round compression function. Validated against: the FIPS 180-4 example vectors (empty message, the string “abc”, and a two-block message).

HMAC-SHA256 (RFC 2104, *crypto/j_hmac.py*). Uses a 64-byte block size, the standard 0x36 inner-pad and 0x5C outer-pad bytes, hashes keys longer than the block size first, and zero-pads shorter keys. Validated against: the RFC 4231 HMAC-SHA256 test cases.

HKDF (extract + expand) (RFC 5869, *crypto/j_hkdf.py*). Built on our own HMAC-SHA256 implementation, providing both the Extract and Expand steps. Validated against: the PRK and OKM values in RFC 5869 Appendix A.

ChaCha20 (RFC 8439, *crypto/a_chacha20.py*). Implements the quarter-round function, the 20-round (10 double-round) permutation, little-endian state serialization, and the per-block counter. Validated against: the RFC 8439 ChaCha20 block-function and keystream test vectors.

Poly1305 (RFC 8439, *crypto/a_poly1305.py* + *crypto/a_poly1305_key_gen.py*). The 16-byte *r* value is clamped as specified, arithmetic is performed modulo $p = 2^{130} - 5$, each 16-byte message block is padded with a high 0x01 byte, and the final tag is produced by adding *s* and reducing modulo 2^{128} . The one-time Poly1305 key is generated by running a ChaCha20 block with counter 0 and taking its first 32 output bytes. Validated against: the 11 RFC 8439 Poly1305 test vectors plus the RFC 8439 key-generation vectors.

ChaCha20-Poly1305 AEAD (RFC 8439 §2.8, *crypto/a_aead_chacha20_poly1305.py*). The Poly1305 MAC input is built as AAD || pad16(AAD) || ciphertext || pad16(ciphertext) || len(AAD) as an 8-byte little-endian integer || len(ciphertext) as an 8-byte little-endian integer. The ciphertext itself is produced with the ChaCha20 keystream starting at block counter 1 (counter 0 is reserved for Poly1305 key generation). On decryption the tag is recomputed and compared; on a mismatch the function returns None, which the caller treats as an authentication failure. Validated against: the RFC 8439 §2.8.2 encryption example and the Appendix A.5 decryption example.

X25519 (RFC 7748, *crypto/a_x25519.py*). Implements the Montgomery ladder over Curve25519, the decodeScalar25519 clamping step, and u-coordinate encoding/decoding. Ephemeral keypair generation (*protocol/a_keypair.py*) draws a 32-byte private key from `os.urandom` and multiplies the base point $u = 9$. Validated against: the RFC 7748 §5.2 test vectors.

5. Protocol Design

5.1 Handshake

The client acts as the initiator and the server as the responder. The handshake proceeds as follows:

- Each party generates an ephemeral X25519 key pair. The two ephemeral public keys are exchanged using a Noise-XX-style message wrapper (*protocol/a_handshake.py*), which here serves only as a transport for the public key its own internal key output is discarded and is not used to secure the channel.
- Once both ephemeral public keys are known, each side independently builds an identical handshake transcript using `build_handshake_transcript()` in *protocol/j_transcript.py*. The transcript concatenates five length-prefixed fields (each as a 4-byte big-endian length followed by the field bytes): the protocol version string “SecureChannel-v1”, the client

identity, the server identity, the client ephemeral public key, and the server ephemeral public key.

- Each side computes $\text{HMAC}(\text{PSK}, \text{SHA-256}(\text{transcript}))$ and sends the resulting tag to the other side. Each side also verifies the tag it receives against its own computation over the same transcript. A mismatch raises an error and aborts the connection this is the only failure in the whole protocol that is fatal to the session (see Section 5.6).

Deviation from the spec wording: the project description specifies $\text{HMAC}(\text{PSK}, \text{transcript})$; our implementation instead computes $\text{HMAC}(\text{PSK}, \text{SHA-256}(\text{transcript}))$. In other words, the implementation authenticates the SHA-256 digest of the transcript rather than the raw transcript bytes. This keeps the HMAC input at a fixed size (32 bytes) regardless of how long the transcript itself is, and the transcript content is still authenticated end-to-end through this PSK-HMAC mechanism.

The PSK itself is loaded from the `SECURECHANNEL_PSK` environment variable on both sides. A non-secret default value is used only so the program can be run out of the box for demonstration; any real deployment must set `SECURECHANNEL_PSK` to a value agreed on out-of-band by the two parties, as required by the project specification.

5.2 Key Schedule

Key derivation is performed by `derive_keys()` in `protocol/j_key_schedule.py`. The X25519 ephemeral-ephemeral shared secret computed during the handshake is used as the HKDF input keying material, and the PSK is used as the HKDF salt. Four independent values are derived, each with a distinct HKDF “info” string so that they cannot be confused with one another even though they share the same salt and input keying material:

Derived value	Length	HKDF info string	Purpose
<code>client_key</code>	32 bytes	“client key”	AEAD key for client → server messages
<code>server_key</code>	32 bytes	“server key”	AEAD key for server → client messages
<code>client_nonce_base</code>	12 bytes	“client nonce”	Nonce base for client → server messages
<code>server_nonce_base</code>	12 bytes	“server nonce”	Nonce base for server → client messages

The result is two fully independent directional keys plus two independent directional nonce bases, so the client-to-server and server-to-client sub-channels never share key or nonce material.

Note on transcript binding: `derive_keys()` also accepts a `transcript_hash` parameter, but the current implementation does not mix it into the HKDF derivation. In the current design, the transcript is authenticated separately by the PSK-HMAC step of Section 5.1, rather than being folded into the key derivation itself.

5.3 Message Format

Every application message has a plaintext, authenticated header (built by `make_header()` in `protocol/j_message.py`) and is carried inside a length-prefixed wire frame (built by `pack_message()` / parsed by `unpack_message()`).

Message header

Field	Size	Description
Protocol version	1 byte	Identifies the protocol version in use.
Message type	1 byte	Application-defined message type.
Sender-ID length	2 bytes (big-endian)	Length, in bytes, of the sender-ID field that follows.
Sender ID	variable	Bytes identifying the sending party.
Sequence number	8 bytes (big-endian)	Strictly increasing per-direction counter, used for nonce construction and replay detection.

Wire frame

Field	Size	Description
Header length	2 bytes (big-endian)	Length of the header field that follows.
Header	variable	The plaintext header above, also passed as AEAD associated data.
Ciphertext length	4 bytes (big-endian)	Length of the ciphertext field that follows.
Ciphertext	variable	ChaCha20 output, same length as the plaintext message body.
Poly1305 tag	16 bytes	Trailing authentication tag over header and ciphertext.

`unpack_message()` validates the declared lengths and splits the frame back into these fields before the header and ciphertext are passed to the AEAD layer.

5.4 Nonce Construction

The 96-bit AEAD nonce is built in `make_constant_and_iv()` (`network/j_client.py`) as constant (4 bytes) || iv (8 bytes), where constant is the first 4 bytes of the relevant directional `nonce_base`, and iv is obtained by interpreting the remaining 8 bytes of the `nonce_base` as a big-endian integer, XOR-ing it with the message's sequence number, and re-encoding the result as 8 bytes. Because the `nonce_base` is derived freshly per handshake (Section 5.2) and the sequence number is strictly increasing and never reused within a direction, every (key, nonce) pair used during a session is unique: the key is fixed per direction for the life of the session, and XOR-ing a never-repeating counter into the IV portion guarantees the nonce itself never repeats for that key. This is the standard requirement for safe ChaCha20-Poly1305 use, and the design satisfies it without needing to store or persist any nonce state beyond the sequence counter that is already required for replay protection.

5.5 Associated Data

The message header (Section 5.3) is the AEAD's associated data (AAD): it is sent and stored in the clear, but it is cryptographically bound to the ciphertext through the Poly1305 tag. `encrypt_message()` passes the header bytes as AAD when producing the ciphertext and tag; `decrypt_message()` feeds the same header bytes back in as AAD when verifying. If even a single byte of the header is altered in transit for example, the sequence number or the sender ID the tag the receiver recomputes will not match the tag in the frame, decryption returns `None`, and the message is rejected as an authentication failure. The header is therefore visible to anyone observing the network (it has to be, so the receiver can identify the message before decrypting it) but cannot be modified undetected.

5.6 Replay Protection

Each receiver maintains a per-direction expected sequence number, starting at 0. `recv_secure()` (`network/j_client.py`) and the equivalent receive loops in `network/a_server.py` and the two GUIs accept a message only if it satisfies both conditions: the AEAD tag verifies over the header and ciphertext, and the sequence number in the header equals the receiver's current expected value. There are three possible outcomes for an incoming application-message frame:

Outcome	Condition	Receiver behavior
Authentication failure	Tag does not verify (tampered header or ciphertext)	Message discarded; expected counter not advanced; connection stays open; status "Authentication failed: message discarded."
Replay / reordering	Tag verifies but sequence $\neq$ expected	Message rejected; expected counter not advanced; connection stays open; status "Replay/reordered message rejected."
Accepted	Tag verifies and sequence $==$ expected	Message delivered to the application; expected counter incremented by 1.

The sender only advances its own sequence counter when it actually sends a message that the protocol expects the receiver to accept an attacker-injected or corrupted frame never consumes a sequence number on the legitimate sender's side. Combined with the rule that a discarded or rejected frame never advances the receiver's counter either, the two sides remain synchronized: one bad frame on the wire does not permanently desynchronize the session, and subsequent genuine, in-order messages continue to be accepted normally.

The handshake authentication check in Section 5.1 remains fatal a bad PSK or a failed transcript HMAC still aborts the connection outright. Only failures at the application-message layer (Phase B) are treated as recoverable, since a single discarded chat message should not end an otherwise-legitimate session, whereas a failed handshake means the two parties never established a trustworthy channel in the first place.

6. Security Analysis

6.1 Properties Achieved

- Confidentiality, integrity, and authenticity of application messages, via ChaCha20-Poly1305 AEAD (Section 4, Section 5.5).
- Mutual authentication and man-in-the-middle resistance, via a PSK-authenticated HMAC over a transcript that binds both parties' ephemeral public keys, both identities, and the protocol version string (Section 5.1). Binding both ephemeral keys specifically prevents an attacker from substituting their own ephemeral key for one side while leaving the other side's key untouched.
- No nonce reuse within a session, by construction of the per-direction nonce base and strictly increasing sequence number (Section 5.4).
- Replay and reordering rejection, via strictly increasing sequence numbers that are bound into both the AEAD nonce and the authenticated header (Sections 5.4–5.6).

6.2 Known Limitations

- Tag and HMAC comparisons currently use Python's "!=" operator, which is not constant-time. This is a timing side channel that a future revision should close with a constant-time comparison.
- Message delivery is required to be strictly in-order: any reordering, even of otherwise-legitimate messages, is rejected rather than buffered. There is no sliding-window mechanism as used in protocols like IPsec or WireGuard.
- The Noise-XX-style wrapper retained in `protocol/a_handshake.py` is used purely as an ephemeral-key transport; its own handshake-completion key material is not part of the trust model and is discarded.

7. Testing

Every primitive is validated in test/ against the official test vectors published in its specification. A transcript-consistency test additionally cross-checks our SHA-256 output against Python's standard-library hashlib implementation. All tests pass.

Primitive	Specification source	Result
SHA-256	FIPS 180-4 example vectors (empty, "abc", two-block message)	Pass
HMAC-SHA256	RFC 4231 test cases	Pass
HKDF (extract + expand)	RFC 5869 Appendix A (PRK and OKM)	Pass
ChaCha20	RFC 8439 block-function / keystream vectors	Pass
Poly1305 (+ key generation)	RFC 8439 Poly1305 vectors and key-generation vectors	Pass

Primitive	Specification source	Result
ChaCha20-Poly1305 AEAD	RFC 8439 §2.8.2 (encrypt) and Appendix A.5 (decrypt)	Pass
X25519	RFC 7748 §5.2 vectors	Pass
SHA-256 cross-check	Comparison against Python hashlib	Pass

8. How to Run

Full setup instructions live in the repository README; the essentials are summarized here.

- Set a shared pre-shared key on both machines before starting a real session: export `SECURECHANNEL_PSK=<agreed secret>`. If unset, the program falls back to a non-secret demo default, intended only for local testing.
- Start the server first: `python network/a_server.py`
- Start the client in a second terminal: `python network/j_client.py`
- Optional Tkinter GUI (runs the same protocol code, not a separate implementation): `python bonus/j_launcher.py`
- Run each test file directly with Python, for example:

```
python test/j_test_sha256.py
python test/j_test_hmac.py
python test/j_test_hkdf.py
python test/j_test_transcript.py

python test/a_test_chacha20.py
python test/a_test_poly1305.py
python test/a_test_poly1305_key_gen.py
python test/a_aead_chacha20_poly1305_enc.py
python test/a_aead_chacha20_poly1305_dec.py
python test/a_testx25519.py
```

9. References

All consulted sources are listed below per the from-scratch rule (consulting a specification or tutorial is permitted; copying an implementation from one is not).

1. National Institute of Standards and Technology (NIST). *FIPS PUB 180-4: Secure Hash Standard (SHS)*. Gaithersburg, MD: NIST, 2015.
2. H. Krawczyk, M. Bellare, and R. Canetti. *RFC 2104: HMAC: Keyed-Hashing for Message Authentication*. Internet Engineering Task Force (IETF), 1997.
3. H. Krawczyk and P. Eronen. *RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. Internet Engineering Task Force (IETF), 2010.
4. Y. Nir and A. Langley. *RFC 8439: ChaCha20 and Poly1305 for IETF Protocols*. Internet Engineering Task Force (IETF), 2018.

5. A. Langley, M. Hamburg, and S. Turner. *RFC 7748: Elliptic Curves for Security*. Internet Engineering Task Force (IETF), 2016.
 6. Christof Paar and Jan Pelzl. *Understanding Cryptography: A Textbook for Students and Practitioners*. 2nd Edition. Springer, 2020.
 7. Trevor Perrin. *The Noise Protocol Framework*. Available at: <https://noiseprotocol.org/noise.html>
-

10. AI Usage Statement

(1) AI tools used: Google Gemini

(2) How it was used: Gemini was used only as a learning and assistance tool, not as an author of project code. Specifically, it was used to explain some cryptographic concepts and algorithms that were unclear while reading the relevant RFC/FIPS specifications, to help us understand parts of the project specification, to answer implementation questions when we did not have enough time to search through documentation ourselves, to suggest standard Python libraries (for example, Tkinter for the GUI), and to provide general debugging guidance and explanations of error messages. Gemini was not used to generate or implement the core cryptographic primitives or the protocol logic

(3) Work completed without AI assistance: All cryptographic primitive implementations, the handshake, the protocol logic, the key schedule, the AEAD secure channel, the networking logic, the integration of all components, and the testing implementation were written entirely by us without AI assistance.

11. Individual Contribution Statements

Team member 1 – Jibreel Bornat 1230872

All the files that starts with J_ , and some edits on amjads code

Team member 2 – Amjad Qaher 1230800

All the files that starts with A_ , and some edits on jibreels code